

Improving Algorithms for Subgraph Homomorphism via Graph Neural Networks

January 24, 2026

Abstract

Homomorphism is an important structure-preserving mapping between two graphs, and is widely used in constraint satisfaction problems, social network analysis, and graph databases querying et. al. Given a graph G and a pattern Q , the subgraph homomorphism problem is to decide whether there exists a mapping h from Q to G such that if there exists an edge from u to v in Q , G also has an edge from $h(u)$ to $h(v)$. Different from the classic isomorphic mapping that is injective, homomorphism may map two distinct vertices in Q to the same vertex in G , and is more challenging than isomorphic mapping. There exist multiple Graph Neural Networks (GNNs) for (subgraph) isomorphism, but few models are developed for subgraph-homomorphism. We develop the first GNN-based framework for subgraph homomorphism. We show that the framework is more expressive than the vanilla GNN, *i.e.*, it can distinguish more graphs that are not homomorphic. We also a generalization bound for the proposed framework. Using real-life and synthetic graphs, we show that our framework is faster for subgraph matching, and its accuracy is also high.

1 Introduction

Graph pattern matching is an important problem in computer science, focusing on identifying instances of a pattern Q within a graph G . This topic has received extensive interest across various fields, *e.g.*, discovering motifs in biological network [1, 2, 3], finding experts in social networks [4, 5, 6], and reasoning on knowledge graph [7, 8, 9].

There exist two semantics for graph pattern matching, namely subgraph isomorphism and homomorphism. (1) The isomorphic semantics demands that the matched instance of Q in G (*i.e.*, a subgraph of G) must be the same as Q . The semantics has been widely studied, *e.g.*, discovering motifs from biological network [1, 2, 3]. (2) Homomorphic semantics, on the other hand, permits that pattern Q can be “embedded” into the identified subgraph of G , allowing the subgraph to be deviated from Q . It has been widely used in graph query languages like Cypher [10] and SPARQL [11]. The difference between

isomorphism and homomorphism lies in the possibility of allowing different vertices in Q can be mapped to the same vertex in Q (see Section 2).

Related work. Spacecraft wiring (cable harness) design is closely related to harness routing in automotive, aircraft, industrial panels, and other constrained 3D environments. Across these domains, the core computational task is to connect multiple terminal pairs (or nets) through a cluttered layout while respecting geometric and engineering rules and minimizing a physical cost (e.g., length, weight, risk). Below we organize the most relevant literature into modeling paradigms and algorithmic families, and highlight the gap addressed by this paper.

3D routing representations and geometric post-processing. Most practical systems first convert a continuous 3D workspace into a discrete routing graph (regular grid, roadmap, or CAD-derived graph), where obstacles remove vertices/edges and engineering preferences are expressed as node/edge penalties (“cost fields”) [12]. In parallel, there is renewed interest in improving the efficiency and geometric quality of the underlying path-search primitives. Kim *et al.* proposed *JPS-Theta** for 3D cable routing, combining Jump Point Search and Theta* to accelerate search while allowing any-angle paths, and further applied B-spline optimization to produce natural, collision-free cable shapes [13]. These works motivate our graph-based abstraction and also suggest a two-stage pipeline (topology first, geometry smoothing later) that is compatible with spacecraft harness design.

Harness topology optimization: multi-branch structures and bundling. Beyond single-cable shortest paths, harness design requires reasoning about the *shared trunk and branching structure*. Karlsson *et al.* formulated an optimization model that simultaneously (i) minimizes total cable length, (ii) rewards shared edges to promote bundling, and (iii) incorporates case-specific 3D environment information; they developed deterministic algorithms based on Lagrangian relaxation and local improvements, producing multiple topology candidates on industrial 3D cases [12]. Complementary heuristic approaches combine informed graph search with meta-heuristics for multi-branch harnesses. For example, Yang *et al.* proposed an A*-ant-colony hybrid for multi-branch cable layout, explicitly modeling routing cost components and showing improved solutions over classic A* or ACO variants [14]. Compared with these harness-routing studies, our spacecraft setting shares the “multi-terminal routing + topology” nature but differs in (i) using *weight* as the primary cost driver and (ii) imposing an explicit global *acyclicity* requirement (tree/forest wiring without redundant loops).

Optimization formulations, hardness, and exact solvers. Many routing and interconnection-design problems can be expressed as variants of Steiner tree/forest, multi-commodity flow, or more general network-design formulations. These formulations are computationally intractable in general: Steiner tree is NP-complete [15], and Steiner-forest-type connectivity objectives remain NP-hard [16]. Karlsson *et al.* also noted that their objective reduces to Steiner tree in a special case, thereby inheriting NP-hardness [12]. For exact (or provably

near-optimal) solutions on moderate instances, mixed-integer linear programming (MILP) with modern solvers remains a strong baseline. General-purpose solvers such as SCIP [17] and commercial optimizers (e.g., COPT [18]) provide branch-and-bound/branch-and-cut engines and can incorporate rich linear constraints. To enforce acyclicity, classic modeling tools include subtour-elimination constraints and ordering-based constraints such as Miller–Tucker–Zemlin (MTZ) [19], as well as flow-based arborescence constraints. Our exact baseline follows this tradition, while also serving as a reference for validating heuristic solutions.

Deterministic heuristics with bounds and multi-solution generation. When the graph or the number of terminal pairs grows, deterministic heuristics are often preferred for speed, controllability, and reproducibility. An effective pattern is *relax-and-repair*: relax difficult coupling constraints (e.g., via Lagrangian relaxation), solve decomposed shortest-path subproblems, and then apply local search/repair to obtain feasible topologies. This is the central idea in [12], where projected subgradient updates on dual multipliers guide the generation of diverse initial solutions, followed by routing heuristics that refine topology and approximate the Pareto front. Our method follows the same high-level philosophy but targets a different feasibility region: we incorporate an explicit acyclicity constraint and design a deterministic procedure that can return the best and near-best (suboptimal) *acyclic* routing topologies for downstream engineering selection.

Learning-assisted routing and learning-based combinatorial optimization. Recent work has explored learning for 3D cable routing in CAD workflows. Kim and Kwon investigated deep reinforcement learning for automatic cable routing in 3D electrical panels, integrating the agent into a CAD loop and reporting competitive path quality [20]. In adjacent “cable route planning” domains, reinforcement learning has also been used for multi-objective submarine cable routing [21]. At a more abstract level, learning-based solvers for Steiner-type problems have progressed via neural-algorithmic hybrids (e.g., combining neural scoring with guided search) [22] and instance-reduction/pruning strategies [23]; benchmarks such as SteBen aim to standardize evaluation for neural Steiner solvers [24]. These learning-based directions are promising for scalability and warm-starting, but often provide limited optimality guarantees. In contrast, our focus is a *traditional*, fully deterministic algorithm that (i) admits an exact MILP formulation for correctness and benchmarking, and (ii) extends to a fast heuristic capable of generating multiple high-quality acyclic candidates.

Cable-specific feasibility constraints: tangling, loops, and topology control. Finally, while many studies emphasize collision avoidance, cable systems also face cable-specific hazards such as snagging, tangling, and undesired loops. For tethered mobile robots, Shimada *et al.* proposed tangle- and contact-free path planning to mitigate entanglement risks [25]. Although our application differs, these results highlight that controlling loop-like behaviors is practically important. Our explicit acyclicity constraint can be viewed as a structural safeguard that rules out redundant loop segments in spacecraft harness

topologies.

Positioning of this work. In summary, existing harness routing literature provides strong foundations in (i) graph-based modeling with environment costs, (ii) MILP formulations with solver baselines, and (iii) deterministic relax-and-repair heuristics for scalable candidate generation. We contribute a spacecraft-oriented formulation that adds a global acyclicity constraint and a deterministic algorithmic framework that unifies: an exact solver-based baseline and a heuristic extension that returns the optimal and near-optimal acyclic routing solutions.

2 Preliminaries

Let Γ be a countably infinite set of labels.

Graphs. Consider directed labeled graphs $G = (V, E, L)$, where (1) V is a finite set of vertices; (2) $E \subseteq V \times \Gamma \times V$ is a finite set of edges, where $\langle v_1, r, v_2 \rangle$ is an edge from v_1 to v_2 , carrying label $r \in \Gamma$; and (3) L is a labeling function that assigns a label $L(v) \in \Gamma$ to each vertex v in V . Let R_G be the set of all edge labels in G .

For each vertex $u \in V$, denote by $N_r^+(u)$ (resp. $N_r^-(u)$) the set of its *outgoing* (resp. *incoming*) neighbors u_1 of u such that there exists an edge labeled r from u to u_1 (resp. from u_1 to u). That is, $N_r^+(u) = \{u_1 \mid \langle u, r, u_1 \rangle \in E\}$ and $N_r^-(u) = \{u_1 \mid \langle u_1, r, u \rangle \in E\}$. Denote by $N(u)$ the set of all neighbors of u , *i.e.*, $N(u) = \bigcup_{r \in R_G} (N_r^-(u) \cup N_r^+(u))$.

Isomorphism and homomorphism. We introduce the semantics of subgraph isomorphism and homomorphism. Consider a pattern $Q = (V_Q, E_Q, L_Q)$ and a graph $G = (V_G, E_G, L_G)$. Here, a pattern is also a directed labeled graph, introduced to simplify the presentation.

Subgraph isomorphism. An *isomorphic mapping* from Q to G , denoted by $Q \sqsubseteq G$ is an injective mapping φ from V_Q to V_G such that (1) for each $u \in V_Q$, u and $\varphi(u)$ carry the same label, *i.e.*, $L_Q(u) = L_G(\varphi(u))$; and (2) the adjacency relation preserves, *i.e.*, $\langle u, r, u' \rangle \in E_Q$ if and only if $\langle \varphi(u), r, \varphi(u') \rangle \in E_G$. When such an injective φ exists, we say that Q is subgraph isomorphic to G .

3 Mathematical Model

We model spacecraft wiring layout as selecting an *acyclic* subgraph that supports simultaneous connections for a set of device pairs. The model is graph-based and is compatible with both exact MILP solving (our baseline) and relaxation-based heuristics (our proposed fast algorithm).

Input graph and terminal pairs. Let $G = (V, E)$ be an undirected graph representing feasible wiring locations (nodes) and allowable cable segments (edges). Each edge $e = \{i, j\} \in E$ has a nonnegative weight w_e capturing physical cost (in our setting, the induced cable weight). The wiring requirements are given as a set of terminal pairs $\mathcal{K} = \{(s_k, t_k)\}_{k=1}^K$. Optionally, each pair can

have a scaling factor ω_k (e.g., different cable gauges), which multiplies the cost contributed by that pair.

Decision variables. We use a multi-commodity flow view to encode one path per required pair and a global edge-selection view to enforce acyclicity.

- $f_{ij}^k \in [-1, 1]$ for each ordered pair (i, j) corresponding to an undirected edge $\{i, j\} \in E$ and each $k \in [K]$. Intuitively, $f_{ij}^k = 1$ means commodity k uses edge $\{i, j\}$ in direction $i \rightarrow j$, and $f_{ij}^k = -1$ means it uses the reverse direction.
- $g_{ij}^k \geq 0$ is an auxiliary variable used to linearize $|f_{ij}^k|$.
- $x_{ij} \in \{0, 1\}$ for each ordered edge (i, j) (one direction chosen if the edge is used globally). The set of globally selected directed edges induces the harness topology.
- $y_i \in [0, 1]$ is an ordering variable used to eliminate directed cycles in the selected topology.

Objective (minimum total weight). We minimize the total weighted usage of edges across all required connections:

$$\min \sum_{k=1}^K \sum_{\{i,j\} \in E} w_{ij} \omega_k g_{ij}^k. \quad (1)$$

Here $w_{ij} = w_{\{i,j\}}$ and g_{ij}^k represents $|f_{ij}^k|$.

Connectivity constraints (one unit flow per pair). We enforce anti-symmetry and flow conservation to ensure each commodity forms an s_k - t_k path:

$$f_{ij}^k = -f_{ji}^k, \quad \forall \{i, j\} \in E, \forall k, \quad (2)$$

$$\sum_{j: \{i,j\} \in E} f_{ij}^k = \begin{cases} 1, & i = s_k, \\ -1, & i = t_k, \\ 0, & \text{otherwise,} \end{cases} \quad \forall i \in V, \forall k. \quad (3)$$

The bounds $-1 \leq f_{ij}^k \leq 1$ are implicit in the variable domain.

Linearization of $|f|$. We replace $|f_{ij}^k|$ by g_{ij}^k using standard linear constraints:

$$g_{ij}^k \geq f_{ij}^k, \quad g_{ij}^k \geq -f_{ij}^k, \quad \forall \{i, j\} \in E, \forall k. \quad (4)$$

Coupling “per-pair routing” with “global topology”. To ensure that an edge used by any commodity must be present in the global topology, we require

$$g_{ij}^k \leq x_{ij} + x_{ji}, \quad \forall \{i, j\} \in E, \forall k. \quad (5)$$

That is, if any pair routes through $\{i, j\}$, at least one direction of this edge must be globally selected.

Acyclicity (no loops in the harness topology). We enforce that the selected directed edges form a forest (no directed cycles). First, we select at most one direction for each undirected edge:

$$x_{ij} + x_{ji} \leq 1, \quad \forall \{i, j\} \in E. \quad (6)$$

Then we impose an ordering-based (MTZ-style) acyclicity constraint:

$$y_i - y_j + x_{ij} \leq 1 - \varepsilon, \quad \forall \{i, j\} \in E, i \neq j, \quad (7)$$

where $\varepsilon > 0$ is a small constant. If $x_{ij} = 1$ then (7) enforces $y_i < y_j$, and hence no directed cycle can exist. In practice we also use degree-style constraints to match engineering requirements (e.g., limiting the number of incoming “parent” edges per node) when needed.

NP-hardness. **Lemma 1:** *The decision version of the spacecraft wiring problem is NP-hard.* $\square$ [Proof sketch] Consider the special case with a single cable type (all $\omega_k = 1$) and replace the terminal-pair set $\mathcal{K}$ by a set of terminals that must be connected within one connected acyclic subgraph. This reduces to the (node-weight-free) Steiner tree problem on graphs, which is NP-complete [15]. More generally, allowing multiple terminal pairs subsumes Steiner forest, which is NP-hard [16]. Therefore, our problem is NP-hard.

Solution extraction. Given an optimal (or near-optimal) solution, for each pair k we recover its route by taking the set of edges with $g_{ij}^k > 0$ (or $|f_{ij}^k| > 0$) and tracing the unique s_k - t_k path. The union of globally selected edges $\{\{i, j\} \mid x_{ij} + x_{ji} = 1\}$ forms an acyclic harness topology that satisfies all pairwise connectivity requirements.

Remark on extensions. Our existing implementation uses a small number of “layers” to model routing choices and additional constraints. The formulation above is the core; adding layers introduces an additional index (e.g., $p \in [P]$) on x and y together with a per-pair layer selection variable, while preserving linearity and the acyclicity mechanism.

4 Theoretical analysis

5 Experimental study

6 Conclusion

In this paper, we proposed a framework **HFrame** for the subgraph homomorphic problem by combining algorithmic solutions and ML models. We showed that

HFrame is more expressive than the vanilla GNN models for subgraph matching. Moreover, we also provided the first generalization error bound for the subgraph matching problem. Using real-life and synthetic graphs, experimental results validate the efficiency and effectiveness of the proposed frameworks.

References

- [1] J. Chen, H. Peng, G. Han, H. Cai, J. Cai, Hogmmnc: a higher order graph matching with multiple network constraints model for gene–drug regulatory modules identification, *Bioinformatics* 35 (4) (2019) 602–610.
- [2] N. Licheri, V. Bonnici, M. Beccuti, R. Giugno, Grapes-dd: exploiting decision diagrams for index-driven search in biological graph databases, *BMC bioinformatics* 22 (2021) 1–24.
- [3] Y. Tian, R. C. Mceachin, C. Santos, D. J. States, J. M. Patel, Saga: a subgraph matching tool for biological graphs, *Bioinformatics* 23 (2) (2007) 232–239.
- [4] Q. Shi, G. Liu, K. Zheng, A. Liu, Z. Li, L. Zhao, X. Zhou, Multi-constrained top-k graph pattern matching in contextual social graphs, in: *ICWS*, 2017, pp. 588–595.
- [5] W. Fan, X. Wang, Y. Wu, Expfinder: Finding experts by graph pattern matching, in: *ICDE*, 2013, pp. 1316–1319.
- [6] J. Brynielsson, J. Högberg, L. Kaati, C. Mrule(a)rtenson, P. Svenson, Detecting social positions using simulation, in: *ASONAM*, 2010, pp. 48–55.
- [7] K. Xu, L. Wang, M. Yu, Y. Feng, Y. Song, Z. Wang, D. Yu, Cross-lingual knowledge graph alignment via graph matching neural network, *arXiv preprint arXiv:1905.11605* (2019).
- [8] K. Teru, E. Denis, W. Hamilton, Inductive relation prediction by subgraph reasoning, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 9448–9457.
- [9] Z. Li, Z. Hou, S. Guan, X. Jin, W. Peng, L. Bai, Y. Lyu, W. Li, J. Guo, X. Cheng, Hismatch: Historical structure matching based temporal knowledge graph reasoning, *arXiv preprint arXiv:2210.09708* (2022).
- [10] N. Dragovic, C. Okulmus, M. Ortiz, Rewriting ontology-mediated navigational queries into cypher, in: *DL*, Vol. 3515, 2023.
- [11] P. Peng, S. Ji, M. T. Özsu, L. Zou, Minimum motif-cut: a workload-aware RDF graph partitioning strategy, *VLDB J.* 33 (5) (2024) 1517–1542.

- [12] T. Karlsson, E. *rule(A)*blad, T. Hermansson, J. S. Carlson, G. Tenfält, Automatic cable harness layout routing in a customizable 3d environment, *Computer-Aided Design* 168 (2024) 103671. doi:10.1016/j.cad.2023.103671.
- [13] K. Kim, Y. Yoon, B. C. Kim, J. Kim, S. Han, S. Kwon, Automatic cable routing based on improved pathfinding algorithm and B-spline optimization for collision avoidance, *Journal of Computational Design and Engineering* 11 (5) (2024) 303–317. doi:10.1093/jcde/qwae085.
- [14] F. Yang, X. Zhao, J. Huang, Z. Liu, A*-ant colony optimization algorithm for multi-branch cable harness layout routing, *Electronics* 13 (3) (2024) 529. doi:10.3390/electronics13030529.
- [15] M. R. Garey, D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, 1979.
- [16] D. P. Williamson, M. X. Goemans, M. Mihail, V. V. Vazirani, A primal-dual approximation algorithm for generalized Steiner network problems, *Combinatorica* 15 (3) (1995) 435–454. doi:10.1007/BF01299747.
- [17] T. Achterberg, *SCIP: Solving constraint integer programs*, *Mathematical Programming Computation* 1 (1) (2009) 1–41. doi:10.1007/s12532-008-0001-1.
- [18] D. Ge, Q. Huangfu, Z. Wang, J. Wu, Y. Ye, Cardinal optimizer (COPT) user guide, *CoRR abs/2208.14314* (2022). doi:10.48550/arXiv.2208.14314.
- [19] C. E. Miller, A. W. Tucker, R. A. Zemlin, Integer programming formulation of traveling salesman problems, *Journal of the ACM* 7 (4) (1960) 326–329. doi:10.1145/321043.321046.
- [20] K. Kim, S. Kwon, Automatic cable routing in 3d electrical panels using deep reinforcement learning, *Korean Journal of Computational Design and Engineering* 30 (1) (2025) 122–135. doi:10.7315/CDE.2025.122.
- [21] Z. Zhao, H. Zhang, Z. Liu, et al., Multi-objective optimization for submarine optical cable route planning with machine-learning assistance, *Journal of Optical Communications and Networking* 16 (10) (2024) 1018–1032.
- [22] A. B. Kahng, K. Yao, Z. Xu, et al., NN-steiner: A mixed neural-algorithmic approach for the rectilinear steiner minimum tree problem, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024. doi:10.1609/aaai.v38i12.29200.
- [23] J. Zhang, D. Tayebi, S. Ray, D. Ajwani, Learning to prune instances of steiner tree problem in graphs, in: *International Network Optimization Conference (INOC)*, 2024.

- [24] Y. Park, et al., Steben: Steiner tree problem benchmark for neural combinatorial optimization, OpenReview preprint (2024).
- [25] R. Shimada, et al., Tangle- and contact-free path planning for a tethered mobile robot using deep reinforcement learning, Frontiers in Robotics and AI 11 (2024). doi:10.3389/frobt.2024.1388634.

A Appendix